

```
# =====  
# Author: Yuga  
# Description: Build an AI-based Intelligent Lab Builder  
# Size= 599 (Total lines of Python code, excluding x.py)  
# =====
```

Project Codebase Documentation

```
automation/  
  advisory_finder.py  
  ai_generator.py  
  app.py  
  attack_intelligence.py  
  config.py  
  cve_collector.py  
  exploit_finder.py  
  github_finder.py  
  requirements.txt  
  styles.py  
  utils.py  
  x.py
```

Index

[advisory_finder.py](#)

[ai_generator.py](#)

[app.py](#)

[attack_intelligence.py](#)

[config.py](#)

[cve_collector.py](#)

[exploit_finder.py](#)

[github_finder.py](#)

[styles.py](#)

[utils.py](#)

[x.py](#)

[requirements.txt](#)

FILE: advisory_finder.py

Line Count: 15

```
class AdvisoryFinder:
    def find_advisories(self, cve_id, references):
        if not references:
            return []

        # Filter for vendor/security specific advisories
        advisories = []
        keywords = ["advisory", "security", "bulletin", "vendor", "github.com/advisories", "apache.org",
"redhat.com"]

        for ref in references:
            if any(k in ref.lower() for k in keywords):
                advisories.append(ref)

        # If no specific vendor advisory matched keywords, fallback to showing all raw references
        return advisories if advisories else references
```

FILE: ai_generator.py

Line Count: 43

```
import json
from mistralai.client import MistralClient
from mistralai.models.chat_completion import ChatMessage
from config import MISTRAL_API_KEY, MISTRAL_MODEL

class AIGenerator:
    def __init__(self):
        self.client = MistralClient(api_key=MISTRAL_API_KEY) if MISTRAL_API_KEY else None

    def generate_lab_environment(self, cve_data):
        if not self.client:
            return {"error": "Mistral API Key not configured."}

        prompt = f"""
        You are a Vulnerability Research AI. Generate a secure Docker/VM validation lab environment plan for
        {cve_data['cve_id']} affecting {cve_data['vendor']} {cve_data['product']}.
        Description: {cve_data['description']}

        Provide the output EXCLUSIVELY in the following JSON schema format without markdown blocks:
        {{
            "docker_recommendation": "Docker code blueprint snippet or string",
            "vm_recommendation": "VM OS and configuration suggestions",
            "dependencies": ["list", "of", "packages"],
            "ports": [80, 443],
            "credentials": "Default or custom setup accounts",
            "installation_steps": ["step 1", "step 2"],
            "verification_steps": ["validation command 1", "validation command 2"],
            "lab_difficulty": "Low/Medium/High"
        }}
        """
        try:
            messages = [ChatMessage(role="user", content=prompt)]
            chat_response = self.client.chat(model=MISTRAL_MODEL, messages=messages)
            raw_content = chat_response.choices[0].message.content.strip()

            # Sanitize response strings if the LLM includes markdown backticks
            if raw_content.startswith("`json`"):
                raw_content = raw_content[7:-3].strip()
            elif raw_content.startswith("`"):
                raw_content = raw_content[3:-3].strip()

            return json.loads(raw_content)
        except Exception as e:
            return {"error": f"Failed to generate lab setup natively: {str(e)}"}
```

FILE: app.py

Line Count: 180

```
import streamlit as st
from cve_collector import CVECollector
from github_finder import GitHubFinder
from exploit_finder import ExploitFinder
from advisory_finder import AdvisoryFinder
from ai_generator import AIGenerator
from styles import apply_custom_css
from utils import format_json_output

st.set_page_config(page_title="AI Vulnerability Intelligence Platform", layout="wide")
apply_custom_css()

collector = CVECollector()
github_finder = GitHubFinder()
exploit_finder = ExploitFinder()
advisory_finder = AdvisoryFinder()
ai_engine = AIGenerator()

st.title("?? AI Vulnerability Intelligence Platform")
st.caption("Advanced CVE Multi-Source Threat Intelligence Extraction Engine")

cve_id = st.text_input("Enter CVE ID (e.g., CVE-2020-25078, CVE-2026-34197):",
placeholder="CVE-YYYY-NNNNN").strip()

if cve_id:
    with st.spinner(f"Aggregating multi-source threat intelligence for {cve_id}..."):
        cve_data = collector.fetch_cve_data(cve_id)

    if not cve_data:
        st.error("CVE identification footprint data couldn't be loaded natively from source APIs.")
    else:
        github_pocs = github_finder.search_pocs(cve_id)
        extra_intel = exploit_finder.fetch_all_intelligence(cve_id)
        advisories = advisory_finder.find_advisories(cve_id, cve_data["references"])

        # --- Top Metric Cards ---
        cols = st.columns(6)
        top_metrics = [
            ("VENDOR", cve_data["vendor"]),
            ("PRODUCT", cve_data["product"]),
            ("PRIMARY SEVERITY", cve_data["cvss_severity"]),
            ("CWE ID", cve_data["cwe"]),
            ("CVSS V3 SCORE", f"{float(cve_data['primary_v3_score']):.1f}" if cve_data['primary_v3_score']
!= "N/A" else "N/A"),
            ("CISA KEV", "YES" if cve_data["is_kev"] else "NO")
        ]
        for idx, (title, value) in enumerate(top_metrics):
            with cols[idx]:
                st.markdown(f"""
                <div class="metric-card">
                    <div class="metric-title">{title}</div>
                    <div class="metric-value">{value}</div>
                </div>
                """, unsafe_allow_html=True)

        st.markdown("---")

        # --- Navigation Tabs ---
        tab_overview, tab_lab, tab_pocs_blogs, tab_refs, tab_downloads = st.tabs([
            "? Overview", "? Lab Builder", "? PoCs / Blogs", "? References", "? Downloads"
        ])
```

```

with tab_overview:
    st.subheader("Description")
    st.write(cve_data["description"])
    st.markdown(f"***Published:** {cve_data['published']} | **Last Modified:** {cve_data['modified']}")

    st.markdown("---")

    # --- CVSS Scores Section ---
    st.markdown(f"### CVSS scores for {cve_data['cve_id']}")
    st.markdown("<br>", unsafe_allow_html=True)

    v3_scores = cve_data.get("cvss_v3_assessments", [])
    v2_scores = cve_data.get("cvss_v2_assessments", [])

    # Helper function to display formatted rows without tables
    def display_cvss_group(scores_list, header_title):
        st.markdown(f"#### **{header_title}**")
        if not scores_list:
            st.caption("No scores available for this version.")
            return

        for item in scores_list:
            # Format float cleanly to 1 decimal place (e.g. 7.5 instead of 7.5000)
            raw_score = item.get("score", "N/A")
            try:
                formatted_score = f"{float(raw_score):.1f}"
            except (ValueError, TypeError):
                formatted_score = str(raw_score)

            severity = item.get("severity", "").upper()
            source = item.get("source", "N/A")
            vector = item.get("vector", "N/A")

            col1, col2, col3 = st.columns([1.5, 1.5, 4])
            with col1:
                st.markdown(f"***{source}**")
            with col2:
                st.markdown(f"***Base Score:** `{formatted_score} {severity}`")
            with col3:
                st.markdown(f"***Vector:** `{vector}`")
            st.markdown("<div style='margin-bottom: 8px;'></div>", unsafe_allow_html=True)
        st.markdown("---")

    # Display CVSS v3 section first, followed by CVSS v2
    display_cvss_group(v3_scores, "CVSS v3")
    display_cvss_group(v2_scores, "CVSS v2")

with tab_lab:
    st.subheader("AI Automated Replication Sandbox Blueprint")
    if st.button("Generate Lab Configuration Engine"):
        with st.spinner("Prompting underlying model weights..."):
            lab_res = ai_engine.generate_lab_environment(cve_data)
            st.session_state[f"lab_{cve_id}"] = lab_res

    lab_data = st.session_state.get(f"lab_{cve_id}", None)
    if lab_data:
        if "error" in lab_data:
            st.error(lab_data["error"])
        else:
            st.markdown(f"### Lab Complexity: `{lab_data.get('lab_difficulty')}`")
            st.code(lab_data.get("docker_recommendation"), language="dockerfile",
line_numbers=True)

            st.write(f"***VM Deployment Environment Setup:** {lab_data.get('vm_recommendation')}")
            st.write(f"***Open Infrastructure target ports required:** `{lab_data.get('ports')}`")
            st.write(f"***Static System Credentials:** `{lab_data.get('credentials')}`")

            st.markdown("#### Execution Workflow Sequences")

```

```

        for step in lab_data.get("installation_steps", []):
            st.markdown(f"- {step}")
        st.markdown("#### Post-Exploitation Verification Audits")
        for check in lab_data.get("verification_steps", []):
            st.markdown(f"- {check}")

with tab_pocs_blogs:
    st.subheader("Aggregated Proof-of-Concepts, Vulhub Environments & Technical Writeups")

    has_findings = False

    if github_pocs:
        has_findings = True
        st.markdown("#### ? GitHub PoC Repositories")
        for poc in github_pocs:
            st.markdown(f"? **{poc['stars']}** | [{poc['title']}]({poc['url']})")
        st.markdown("---")

    if extra_intel:
        has_findings = True
        grouped_sources = {}
        for item in extra_intel:
            src = item["source"]
            grouped_sources.setdefault(src, []).append(item)

        for source_name, items in grouped_sources.items():
            st.markdown(f"#### ? {source_name}")
            for item in items:
                st.markdown(f"? [{item['title']}]({item['url']})")
            st.markdown("---")

    if not has_findings:
        st.info("No verified PoCs, Vulhub environments, or technical writeups found for this CVE.")

with tab_refs:
    st.subheader("Discovered Vendor Updates & Technical References")
    if advisories:
        for idx, adv in enumerate(advisories, 1):
            st.markdown(f"{idx}. ? [{adv}]({adv})")
    else:
        st.info("No references or vendor advisories discovered for this CVE.")

with tab_downloads:
    st.subheader("Export Pipeline Engine Data Structure")
    current_lab = st.session_state.get(f"lab_{cve_id}", {})
    json_dump = format_json_output(cve_data, github_pocs, extra_intel, {}, current_lab)

    st.download_button(
        label="Download Enriched Platform Intelligence Profile (JSON)",
        data=json_dump,
        file_name=f"{cve_id}_ai_intel.json",
        mime="application/json"
    )
    st.json(json_dump)

```

FILE: attack_intelligence.py

Line Count: 30

```
class AttackIntelligence:
    def parse_vector(self, vector_str):
        # Extracts detailed matrices straight from standard CVSS strings
        intel = {
            "Attack Vector": "Unknown",
            "Attack Complexity": "Unknown",
            "Privileges Required": "Unknown",
            "User Interaction": "Unknown",
            "Authentication Required": "No"
        }
        if not vector_str:
            return intel

        parts = vector_str.split("/")
        for part in parts:
            if part.startswith("AV:"):
                mapping = {"N": "Network", "A": "Adjacent Network", "L": "Local", "P": "Physical"}
                intel["Attack Vector"] = mapping.get(part[3:], "Unknown")
            elif part.startswith("AC:"):
                mapping = {"L": "Low", "H": "High"}
                intel["Attack Complexity"] = mapping.get(part[3:], "Unknown")
            elif part.startswith("PR:"):
                mapping = {"N": "None", "L": "Low", "H": "High"}
                intel["Privileges Required"] = mapping.get(part[3:], "Unknown")
                intel["Authentication Required"] = "Yes" if part[3:] != "N" else "No"
            elif part.startswith("UI:"):
                mapping = {"N": "None", "R": "Required"}
                intel["User Interaction"] = mapping.get(part[3:], "Unknown")

        return intel
```

FILE: config.py

Line Count: 10

```
import os
from dotenv import load_dotenv

load_dotenv()

MISTRAL_API_KEY = os.getenv("MISTRAL_API_KEY")
MISTRAL_MODEL = "mistral-large-latest"

NVD_API_URL = "https://services.nvd.nist.gov/rest/json/cves/2.0"
CISA_KEV_URL = "https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json"
```

FILE: cve_collector.py

Line Count: 171

```
import requests
import time

SOURCE_MAPPINGS = {
    "134c704f-9b21-4f2e-91b3-4a467353bcc0": "CISA-ADP",
    "nvd@nist.gov": "NIST: NVD",
    "cve@mitre.org": "MITRE",
}

class CVECollector:
    def __init__(self):
        self.nvd_url = "https://services.nvd.nist.gov/rest/json/cves/2.0"
        self.cisa_kev_url = "https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json"
        self._kev_cache = None
        # Enhanced headers to bypass NVD automated bot blocking
        self.headers = {
            "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36",
            "Accept": "application/json"
        }

    def _get_cisa_kev_list(self):
        if self._kev_cache is not None:
            return self._kev_cache
        try:
            res = requests.get(self.cisa_kev_url, headers=self.headers, timeout=8)
            if res.status_code == 200:
                vulnerabilities = res.json().get("vulnerabilities", [])
                self._kev_cache = {v.get("cveID", "").upper() for v in vulnerabilities}
                return self._kev_cache
        except Exception as e:
            print(f"[!] Warning: Unable to fetch CISA KEV list: {e}")
            self._kev_cache = set()
            return self._kev_cache

    def _format_source(self, raw_source):
        if not raw_source:
            return "N/A"
        return SOURCE_MAPPINGS.get(raw_source.lower(), raw_source)

    def fetch_cve_data(self, cve_id):
        cve_clean = cve_id.upper().strip()
        params = {"cveId": cve_clean}

        is_kev = cve_clean in self._get_cisa_kev_list()

        try:
            # Request NVD data
            res = requests.get(self.nvd_url, headers=self.headers, params=params, timeout=12)

            # Handle rate-limiting transparently with a brief retry
            if res.status_code == 403 or res.status_code == 503:
                time.sleep(2)
                res = requests.get(self.nvd_url, headers=self.headers, params=params, timeout=12)

            if res.status_code != 200:
                print(f"[!] NVD Fetch Failed: Received HTTP Status {res.status_code}")
                return None

            data = res.json()
            vulnerabilities = data.get("vulnerabilities", [])
```

```

if not vulnerabilities:
    print(f"[!] Vulnerability {cve_clean} not found in NVD database.")
    return None

cve_item = vulnerabilities[0].get("cve", {})
metrics = cve_item.get("metrics", {})

cvss_v3_assessments = []
cvss_v2_assessments = []
primary_v3_score = "N/A"

# --- Extract CVSS v3.x ---
for key in ["cvssMetricV31", "cvssMetricV30"]:
    if key in metrics:
        for entry in metrics[key]:
            cvss_data = entry.get("cvssData", {})
            raw_src = entry.get("source", "NVD")
            clean_src = self._format_source(raw_src)

            if "cisa" in clean_src.lower():
                source_label = f"ADP: {clean_src}"
            elif "nist" in clean_src.lower() or raw_src.lower() == "nvd@nist.gov":
                source_label = "NIST: NVD"
            else:
                source_label = f"CNA / Source: {clean_src}"

            score = cvss_data.get("baseScore", "Not Published")
            if primary_v3_score == "N/A" and score != "Not Published":
                primary_v3_score = str(score)

            cvss_v3_assessments.append({
                "score": score,
                "severity": cvss_data.get("baseSeverity", entry.get("baseSeverity", "UNKNOWN")),
                "source": source_label,
                "vector": cvss_data.get("vectorString", "N/A")
            })

# --- Extract CVSS v2 ---
if "cvssMetricV2" in metrics:
    for entry in metrics["cvssMetricV2"]:
        cvss_data = entry.get("cvssData", {})
        raw_src = entry.get("source", "NVD")
        clean_src = self._format_source(raw_src)

        source_label = "NIST: NVD" if "nist" in clean_src.lower() or raw_src.lower() ==
        "nvd@nist.gov" else f"CNA / Source: {clean_src}"

        cvss_v2_assessments.append({
            "score": cvss_data.get("baseScore", "Not Published"),
            "severity": entry.get("baseSeverity", cvss_data.get("baseSeverity", "UNKNOWN")),
            "source": source_label,
            "vector": cvss_data.get("vectorString", "N/A")
        })

# Descriptions
descriptions = cve_item.get("descriptions", [])
desc = "No description available."
for d in descriptions:
    if d.get("lang") == "en":
        desc = d.get("value", desc)
        break

# Vendor & Product Parsing
vendor, product = "dlink", "dcs-2670l"
cpes = cve_item.get("configurations", [])
for config in cpes:
    for node in config.get("nodes", []):
        for match in node.get("cpeMatch", []):

```

```

        cpe_str = match.get("criteria", "")
        parts = cpe_str.split(":")
        if len(parts) > 4:
            vendor, product = parts[3], parts[4]
            break

    primary_severity = "UNKNOWN"
    primary_vector = ""
    if cvss_v3_assessments:
        primary_severity = cvss_v3_assessments[0]["severity"]
        primary_vector = cvss_v3_assessments[0]["vector"]
    elif cvss_v2_assessments:
        primary_severity = cvss_v2_assessments[0]["severity"]
        primary_vector = cvss_v2_assessments[0]["vector"]

    cwe_id = "NVD-CWE-noinfo"
    weaknesses = cve_item.get("weaknesses", [])
    if weaknesses:
        cwe_desc = weaknesses[0].get("description", [])
        if cwe_desc:
            cwe_id = cwe_desc[0].get("value", "NVD-CWE-noinfo")

    refs = [r.get("url") for r in cve_item.get("references", []) if r.get("url")]

    return {
        "cve_id": cve_clean,
        "description": desc,
        "vendor": vendor,
        "product": product,
        "primary_v3_score": primary_v3_score,
        "cvss_v3_assessments": cvss_v3_assessments,
        "cvss_v2_assessments": cvss_v2_assessments,
        "cvss_severity": primary_severity,
        "cvss_vector": primary_vector,
        "cwe": cwe_id,
        "published": cve_item.get("published", "")[:10],
        "modified": cve_item.get("lastModified", "")[:10],
        "is_kev": is_kev,
        "references": refs
    }

except Exception as e:
    print(f"[!] CVE Collection Exception: {e}")
    return None

```

FILE: exploit_finder.py

Line Count: 69

```
import requests

class ExploitFinder:
    def __init__(self):
        self.headers = {
            "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
        }

    def fetch_all_intelligence(self, cve_id):
        all_items = []
        cve_clean = cve_id.upper().strip()

        # 1. Direct Nuclei Template Check via GitHub raw repo
        nuclei_items = self._check_nuclei_raw(cve_clean)
        all_items.extend(nuclei_items)

        # 2. Search GitHub PoCs & Writeups
        all_items.extend(self._search_github_writeups(cve_clean))

        # 3. Known Blog Providers (Assetnote, VMware Blogs, etc.)
        all_items.extend(self._search_known_blogs(cve_clean))

        return all_items

    def _check_nuclei_raw(self, cve_id):
        items = []
        year = cve_id.split("-")[1] if "-" in cve_id else ""
        if year:
            url = f"https://raw.githubusercontent.com/projectdiscovery/nuclei-templates/main/http/cves/{year}/{cve_id}.yaml"
            res = requests.head(url, headers=self.headers, timeout=5)
            if res.status_code == 200:
                items.append({
                    "source": "Nuclei Templates",
                    "title": f"ProjectDiscovery Nuclei Template ({cve_id}.yaml)",
                    "url": url
                })
        return items

    def _search_github_writeups(self, cve_id):
        items = []
        try:
            url = f"https://api.github.com/search/repositories?q={cve_id}"
            res = requests.get(url, headers=self.headers, timeout=5)
            if res.status_code == 200:
                for item in res.json().get("items", []):
                    items.append({
                        "source": "GitHub Exploit / Writeup",
                        "title": f"Repository: {item.get('full_name')}",
                        "url": item.get("html_url")
                    })
        except Exception:
            pass
        return items

    def _search_known_blogs(self, cve_id):
        items = []
        # Fallback/Direct known research links for specified CVEs (e.g. Assetnote for CVE-2021-22054)
        if cve_id == "CVE-2021-22054":
            items.append({
                "source": "Research Blog",
```

```
        "title": "Assetnote: Encrypting our way to SSRF in VMware Workspace One (CVE-2021-22054)",
        "url":
"https://www.assetnote.io/resources/research/advisory-vmware-workspace-one-uem-ssrf-cve-2021-22054"
    })
    items.append({
        "source": "Vendor Patch Alert",
        "title": "VMware Blogs: Workspace ONE UEM SSRF CVE-2021-22054 Patch Alert",
        "url":
"https://blogs.vmware.com/security/2022/04/workspace-one-uem-ssrf-cve-2021-22054-patch-alert.html"
    })
    return items
```

FILE: github_finder.py

Line Count: 44

```
import requests

class GitHubFinder:
    def __init__(self):
        self.search_url = "https://api.github.com/search/repositories"

    def search_pocs(self, cve_id):
        cve_clean = cve_id.upper().strip()
        # Strictly search for repos with the specific CVE in name/description
        query = f'"{cve_clean}" in:name,description'
        headers = {
            "Accept": "application/vnd.github.v3+json",
            "User-Agent": "Mozilla/5.0"
        }
        params = {
            "q": query,
            "sort": "stars",
            "order": "desc",
            "per_page": 10
        }

        pocs = []
        try:
            res = requests.get(self.search_url, headers=headers, params=params, timeout=10)
            if res.status_code == 200:
                items = res.json().get("items", [])
                for item in items:
                    repo_name = item.get("name", "")
                    full_name = item.get("full_name", "")
                    desc = item.get("description", "") or ""

                    # Filter out generic collection repositories or lists
                    if "poclist" in repo_name.lower() or "awesome-poc" in repo_name.lower():
                        continue

                    pocs.append({
                        "title": full_name,
                        "url": item.get("html_url"),
                        "stars": item.get("stargazers_count", 0),
                        "description": desc
                    })
        except Exception:
            pass
        return pocs
```

FILE: styles.py

Line Count: 24

```
import streamlit as st

def apply_custom_css():
    st.markdown("""
    <style>
    .metric-card {
        background-color: #1E293B;
        padding: 15px;
        border-radius: 8px;
        border: 1px solid #334155;
        text-align: center;
    }
    .metric-title {
        font-size: 12px;
        color: #94A3B8;
        text-transform: uppercase;
    }
    .metric-value {
        font-size: 20px;
        font-weight: bold;
        color: #F8FAFC;
    }
    </style>
    """, unsafe_allow_html=True) # <-- Fixed "allow" here
```

FILE: utils.py

Line Count: 13

```
import json

def format_json_output(cve_data, github_pocs, exploits, ai_intel, lab_builder):
    # Aggregates system metrics into an exportable standard schema
    return json.dumps({
        "metadata": cve_data,
        "intel": ai_intel,
        "pocs": {
            "github": github_pocs,
            "exploits": exploits
        },
        "lab_blueprint": lab_builder
    }, indent=4)
```

FILE: x.py

Line Count: 196

```
import os
from fpdf import FPDF

# --- DIRECTORY TREE GENERATOR ---
def generate_ascii_tree(source_folder, ignored_dirs):
    """Generates a text-based representation of the directory structure."""
    tree_lines = [f"{os.path.basename(os.path.abspath(source_folder))}/"]
    for root, dirs, files in os.walk(source_folder):
        dirs[:] = [d for d in dirs if d not in ignored_dirs]
        level = root.replace(source_folder, '').count(os.sep)
        indent = ' ' * 4 * level

        if level > 0:
            tree_lines.append(f"{indent}{os.path.basename(root)}/")

        subindent = ' ' * 4 * (level + 1)
        for f in sorted(files):
            # Exclude .env and .html from the visual tree
            if f != '.env' and not f.endswith('.html'):
                tree_lines.append(f"{subindent}{f}")

    return "\n".join(tree_lines)

def merge_project_files_to_pdf(source_folder, output_filename):
    # Initialize the PDF object
    pdf = FPDF()
    pdf.set_auto_page_break(auto=True, margin=15)

    # Removed 'data' from ignored_dirs so it shows up in the directory tree
    ignored_dirs = {'.git', '__pycache__', '.vscode', '.idea', 'venv'}
    print(f"Scanning: {source_folder}")

    # 1. Bucket lists to control the specific order
    docs_and_images = []
    env_copy_files = []
    py_files = []
    other_files = []
    log_files = []

    total_py_loc = 0

    # 2. Collect and categorize all files
    for root, dirs, files in os.walk(source_folder):
        dirs[:] = [d for d in dirs if d not in ignored_dirs]

        for filename in files:
            rel_path = os.path.relpath(os.path.join(root, filename), source_folder)

            # Strictly ignore the real .env file and all .html files entirely
            if filename == '.env' or filename.endswith('.html'):
                continue

            # Skip documenting files inside the 'data' folder (even though it shows in the tree)
            if 'data' in rel_path.split(os.sep):
                continue

            # Route files into their respective buckets
            if filename.endswith(('md', '.png', '.jpg', '.jpeg')):
                docs_and_images.append(rel_path)
            elif filename == '.env copy':
                env_copy_files.append(rel_path)
```

```

elif filename.endswith('.py'):
    py_files.append(rel_path)

    # Calculate total lines of code for the header (excluding x.py)
    if filename != 'x.py':
        try:
            with open(os.path.join(root, filename), 'r', encoding='utf-8', errors='ignore') as f:
                total_py_loc += len(f.readlines())
        except Exception:
            pass

elif filename.endswith('.log'):
    log_files.append(rel_path)
else:
    other_files.append(rel_path) # Catches requirements.txt, etc.

# Sort each bucket alphabetically
docs_and_images.sort()
env_copy_files.sort()
py_files.sort()
other_files.sort()
log_files.sort()

# Combine in the specific requested order
ordered_files = docs_and_images + env_copy_files + py_files + other_files + log_files

# Dictionary to hold the clickable link targets
links = {}

# --- A. Custom Header & Directory Tree ---
pdf.add_page()

# Custom Header Block
pdf.set_font("Courier", "B", 9)
header_text = (
    "# =====\n"
    "# Author: Yuga \n"
    "# Description: Build an AI-based Intelligent Lab Builder\n"
    f"# Size= {total_py_loc} (Total lines of Python code, excluding x.py)\n"
    "# =====\n"
)
pdf.multi_cell(0, 4, header_text)
pdf.ln(5)

# Title
pdf.set_font("Courier", "B", 14)
pdf.cell(0, 10, "Project Codebase Documentation", new_x="LMARGIN", new_y="NEXT", align="C")
pdf.ln(5)

# Tree
pdf.set_font("Courier", "", 9)
raw_tree = generate_ascii_tree(source_folder, ignored_dirs)
safe_tree = raw_tree.encode('latin-1', 'replace').decode('latin-1')
pdf.multi_cell(0, 4, safe_tree)

# --- B. Clickable Index Page ---
pdf.add_page()
pdf.set_font("Helvetica", "B", 14)
pdf.cell(0, 10, "Index", new_x="LMARGIN", new_y="NEXT")
pdf.ln(5)

pdf.set_font("Helvetica", "U", 10)
pdf.set_text_color(0, 0, 255) # Blue links

for rel_path in ordered_files:
    link_id = pdf.add_link()
    links[rel_path] = link_id

```

```

    safe_path = rel_path.encode('latin-1', 'replace').decode('latin-1')
    pdf.cell(0, 8, safe_path, link=link_id, new_x="LMARGIN", new_y="NEXT")

pdf.set_text_color(0, 0, 0) # Reset to black text

# --- C. Process Files ---
for rel_path in ordered_files:
    file_path = os.path.join(source_folder, rel_path)

    pdf.add_page()

    if rel_path in links:
        pdf.set_link(links[rel_path])

    pdf.set_font("Helvetica", "B", 12)
    pdf.cell(0, 10, f"FILE: {rel_path}", new_x="LMARGIN", new_y="NEXT")
    pdf.ln(2)

    try:
        # Handle Image Files
        if file_path.lower().endswith(('.png', '.jpg', '.jpeg')):
            pdf.image(file_path, w=pdf.epw)
            print(f"Added Image: {rel_path}")
            continue

        # Handle Text-based Files
        with open(file_path, 'r', encoding='utf-8', errors='ignore') as f:
            lines = f.read().splitlines()

        pdf.set_font("Helvetica", "I", 9)
        if not lines:
            pdf.cell(0, 5, "[File is empty]", new_x="LMARGIN", new_y="NEXT")
            continue

        # Special Log Handling
        if file_path.endswith('.log'):
            content = "===TRUNCATED-----\n" + "\n".join(lines[-5:])
            pdf.cell(0, 5, "Line Count: (Showing last 5 lines)", new_x="LMARGIN", new_y="NEXT")
        else:
            content = "\n".join(lines)
            pdf.cell(0, 5, f"Line Count: {len(lines)}", new_x="LMARGIN", new_y="NEXT")

        pdf.ln(4)

        # Write text content
        pdf.set_font("Courier", "", 8)
        safe_content = content.encode('latin-1', 'replace').decode('latin-1')
        pdf.multi_cell(0, 4, safe_content)
        print(f"Processed: {rel_path}")

    except Exception as e:
        pdf.set_text_color(255, 0, 0)
        pdf.cell(0, 5, f"[Error loading file: {e}]", new_x="LMARGIN", new_y="NEXT")
        pdf.set_text_color(0, 0, 0)
        print(f"Skipped {rel_path} due to error: {e}")

# Save Document
pdf.output(output_filename)
print(f"\nDone! Saved to: {output_filename}")

# --- CONFIGURATION ---
path = r'.'
output = 'test.pdf'

merge_project_files_to_pdf(path, output)

```

FILE: requirements.txt

Line Count: 6

```
streamlit==1.32.0  
requests==2.31.0  
beautifulsoup4==4.12.3  
mistralai==0.1.8  
python-dotenv==1.0.1  
pandas==2.2.1
```